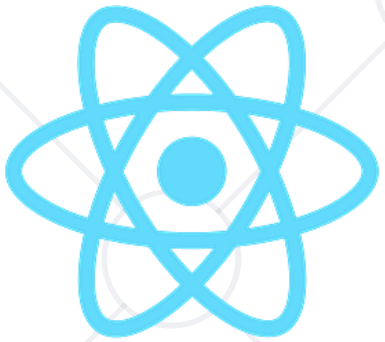


Authentication Build and Deploy



React Native

SoftUni Team
Technical Trainers



SoftUni



Software University

<https://softuni.bg>

Table of Contents

1. Development Setup
2. Build
3. Deployment
4. Distribution
5. Firebase Authentication and Services





Development Setup

Environment Overview

- 
- Why can't we just use Expo Go and skip setup?
 - Because real apps need **more** than previewing JavaScript.
 - Expo Go is a **sandbox**
 - **Limited** native capabilities
 - **Cannot** test production builds
 - **No control** over signing & releases
 - Custom native modules may **fail**
 - Store deployment requires **full toolchain**

Expo Go is for learning and quick iteration — not for shipping.

What Development Setup Gives You

- 
- Professional capability
 - Build **real** binaries (APK / AAB)
 - Install **custom** native dependencies
 - Control environments
 - Debug native errors
 - Prepare for **store submission**

If you want to publish → you need the setup.

- JavaScript runs **outside** the browser
 - **Executes** build tools
 - Runs **Metro bundler**
 - **Installs** dependencies
 - **Powers** scripts
 - **Required** for everything
 - Use **LTS** version



Wrong Node version = weird errors.

- Android apps require **Java tooling**
 - Needed for Gradle builds
 - Required by Android SDK
 - Powers native compilation
 - Must match supported versions
 - Set via JAVA_HOME



Without JDK → no Android build.

- The Android toolbox
 - Emulator manager
 - SDK installer
 - Platform tools
 - Logcat debugging
 - Device configuration



You don't need to write Java —but you need its infrastructure.

Android Studio is mission control.

- Your virtual test device
 - Created via AVD Manager
 - Simulates real hardware
 - Different OS versions
 - No physical phone needed
 - Slower than real devices

```
npm run android
```

Good for development, not perfect for reality.

- Real hardware, real behavior
 - Enable **Developer Options**
 - Turn on **USB debugging**
 - Connect via **cable** or **Wi-Fi**
 - Better performance metrics
 - Essential before release

Emulators lie. Devices don't.

- Apps talk to **different** backends
 - Dev / staging / production
 - API URLs change
 - Keys change
 - Never hardcode secrets
 - Use **.env** files

```
API_URL=https://api.myapp.com
```

Configuration is part of architecture.



Build

What Is a Build?

- A build **converts** code into an installable app
 - JavaScript bundled
 - Assets optimized
 - Native binaries created
 - Signed for stores
 - Environment injected

Build = preparation for users.



- What do we actually produce after a build?
 - Android → APK or AAB
 - iOS → IPA
 - Installable on real devices
 - Uploadable to stores
 - Contains JS + native code + assets

Your app becomes a distributable product.

Development vs Production Builds

- They are very different animals
 - Dev → **fast, debug friendly**
 - Prod → **optimized, minified**
 - Dev has **warnings**
 - Prod has **performance**
 - Always **test** production builds

Never trust dev behavior.

- Build directly on **your machine**
 - Uses installed **SDKs**
 - Full control
 - Useful for debugging
 - Can be slow
 - Setup sensitive

Local builds expose real problems early.

- Cloud builds made **simple**
 - No local native setup
 - iOS + Android
 - Handles certificates
 - Profiles for environments
 - CI/CD friendly

```
eas build --profile production
```

EAS removes 80% of mobile pain.

- Metadata matters
 - App name
 - Bundle identifier
 - Version & build number
 - Icons & splash
 - Permissions

Stores reject sloppy configuration.

- **First impressions** are part of engineering
 - **Required** by stores
 - Multiple sizes needed
 - Platform guidelines apply
 - Generated during build
 - Must match brand identity

Users judge quality in milliseconds.

- Users **upgrade**, not reinstall
 - Increase build number every release
 - Semantic versioning helps
 - Required for store submission
 - Track what changed
 - Vital for debugging

Versioning is communication.



Deployment

What Is Deployment?

- Deployment makes the build **reachable**
 - Upload to store
 - Release to testers
 - Publish updates
 - Rollout gradually
 - Monitor health

Code exists only when users can download it.



- Strict but premium
 - Manual review
 - Strong **UI** expectations
 - Privacy rules serious
 - Certificates **mandatory**
 - Slower approvals



Quality bar is high.

- Faster, **more flexible**
 - Automated checks
 - Closed / open testing tracks
 - **Easier** iteration
 - Still policy heavy
 - **Faster** releases

Speed is your advantage.



- Your marketing page
 - Screenshots
 - Feature graphics
 - App description
 - Keywords
 - Support URL

Users install based on this.

OTA Updates (Over-The-Air)

- Ship **JS updates** without store review
 - Push bug fixes **instantly**
 - Great for **UI tweaks**
 - Cannot change native code
 - Requires **careful** testing
 - Huge productivity boost

Powerful but dangerous if abused.



Distribution

- Start small
 - Team devices
 - QA validation
 - Quick iterations
 - Catch obvious bugs
 - Validate flows

Never surprise real users.

- Real users, limited risk
 - Invite lists
 - Feedback loops
 - Crash discovery
 - Performance validation
 - Usability insights

Your best improvements come from here.

- How **updates** reach users
 - Big bang release
 - Gradual rollout
 - Region based
 - Feature flags
 - Kill switches

Control risk, always.

- Launch is day one
 - Track crashes
 - Monitor performance
 - Watch reviews
 - Measure engagement
 - Patch fast

Shipping starts the real work.

- Install your app without the store.
 - Direct APK installation
 - No store review required
 - Useful for internal testing

Great for development. Not for mass users.



Firebase Authentication and Services

- Backend power without backend teams
 - Authentication
 - Firestore database
 - Cloud functions
 - Analytics
 - Push notifications

Firebase accelerates startups.

- Handles identity management
 - Secure
 - Battle tested
 - Easy SDK
 - Manages tokens
 - Supports providers

```
signInWithEmailAndPassword(auth, email, pass);
```

Don't build auth yourself.

- Backend logic **without managing servers**
 - Run code in the **cloud**
 - Triggered by events or **HTTP**
 - Scales **automatically**
 - Pay for execution
 - Integrates with other **Firebase** services

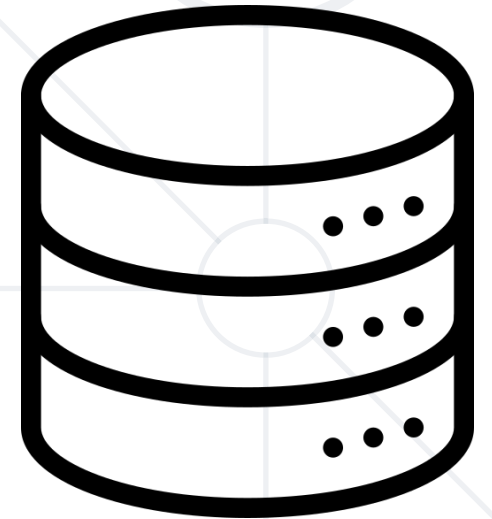
Write backend power with frontend speed.

- Realtime cloud database
 - NoSQL
 - Realtime listeners
 - Offline support
 - Scales automatically
 - Simple API

Great for rapid products.



- Store and serve user files from the cloud
 - **Upload** images, videos, documents
 - Built on **Google Cloud** infrastructure
 - Secure via authentication rules
 - Scales **automatically**
 - Designed for **mobile networks**



Perfect for user-generated content.

- Moving **beyond** Expo Go to a full local or cloud (EAS) environment is essential for custom native modules and store deployment.
- **Reliable builds** require specific versions of Node.js, JDK, and Android Studio to transform JavaScript into installable binaries (APK).
- **Success** depends on proper app configuration, environment variables, and rigorous testing on physical devices rather than just emulators.
- **Publishing** involves managing store assets, utilizing OTA updates for quick fixes, and leveraging Firebase for rapid backend integration (Auth, DB, and Storage).



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

